



Sant Dnyaneshwar Shikshan Sanstha's
**Annasaheb Dange College of Engineering and
Technology (ADCET), Ashta**

Department of CSE (IOT and CSBT)
Quality Laboratory Manual
Data Structures Lab
2ICPC202

Second Year

Sem-III

AY 2026-27

Course Instructor
Mrs. Prachi S. Pathak
ASSISTANT PROFESSOR

Experiment No. 1

Title of Experiment: To study and implement structures using static and dynamic memory allocation along with functions in C.

Aim of Experiment: To understand and implement the concept of structures, memory allocation (static and dynamic), and functions with argument passing and return values in C.

System Requirements:

Programming Languages: 1. C 2. C++ | Programming Editor: 1. Dev C++ 2. GCC/G++

Hardware: 1. A computer that can execute C/C++ programs. | Operating system: 1. Windows / UNIX Operating System.

Experiment Outcomes –

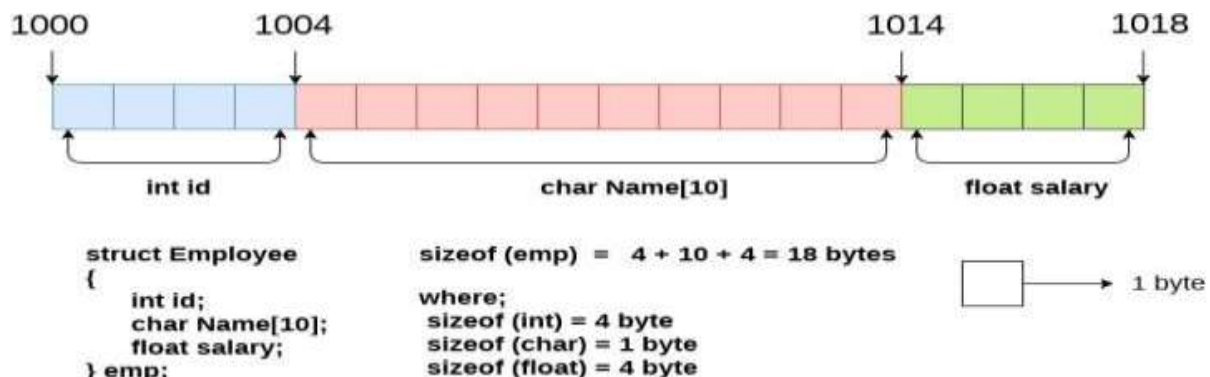
1. Understand the principles of structures and their applications in managing grouped data.
2. Understand and implement memory allocation methods: static and dynamic.
3. Implement and utilize functions with parameters and return types.
4. Achieve modular and reusable code using structure-function integration.

Theory –

1. Structures: A structure is a user defined data type in C/C++. A structure creates a data type that can be used to group items of possibly different types into a single type. *Structure* keyword is used to create a structure.

Syntax of structure:

```
struct structure_name
{
    data_type member1;
    data_type member2;
    ...
    data_type memberN;
};
```



2. Arrays of Structures:

You can create arrays of structures, which is useful when you need to handle multiple records of the same type.

Example:

```
struct Person people[10];
```

This declares an array of 10 *Person* structures. Each element in the array can be accessed like *people[0]*, *people[1]*, etc., and you can manipulate the members of each structure using the dot operator.

Example Usage:

```
people[0].age = 30;  
strcpy(people[0].name, "Alice");  
people[0].height = 5.7;
```

3. Pointers to Structures

Pointers to structures are used to efficiently pass structures to functions or to dynamically allocate memory for them.

Example:

```
struct Person *p;  
p = (struct Person *)malloc(sizeof(struct Person));
```

Here, *p* is a pointer to a *Person* structure, and *malloc* allocates memory for one *Person* structure. You can access the members of the structure through the pointer using the arrow operator (*->*).

Example Usage:

```
p->age = 25;  
strcpy(p->name, "Bob");  
p->height = 6.0;
```

4. Dynamic Allocation

Dynamic memory allocation allows you to create structures and arrays at runtime, which is useful when the size of the data is not known at compile time.

Example of Dynamic Array of Structures:

```
int n = 5;  
struct Person *people = (struct Person *)malloc(n * sizeof(struct Person));
```

Here, *people* is a pointer to an array of 5 *Person* structures, allocated dynamically.

Accessing Members:

```
people[0].age = 40;  
strcpy(people[0].name, "Charlie");  
people[0].height = 5.9;
```

A **function in C** is a set of statements that when called perform some specific task. It is the basic building block of a C program that provides modularity and code reusability. The programming statements of a function are enclosed within **{ } braces**, having certain meanings and performing certain operations. They are also called subroutines or procedures in other languages.

In this article, we will learn about functions, function definition, declaration, arguments and parameters, return values, and many more.

Syntax of Functions in C

The syntax of function can be divided into 3 aspects:

1. Function Declaration
2. Function Definition
3. Function Calls

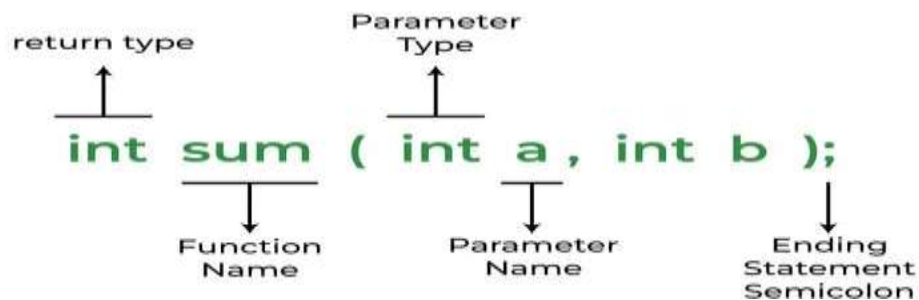
Function Declarations

In a function declaration, we must provide the function name, its return type, and the number and type of its parameters. A function declaration tells the compiler that there is a function with the given name defined somewhere else in the program.

Syntax

return_type name_of_the_function (parameter_1, parameter_2);

The parameter name is not mandatory while declaring functions. We can also declare the function without using the name of the data variables.

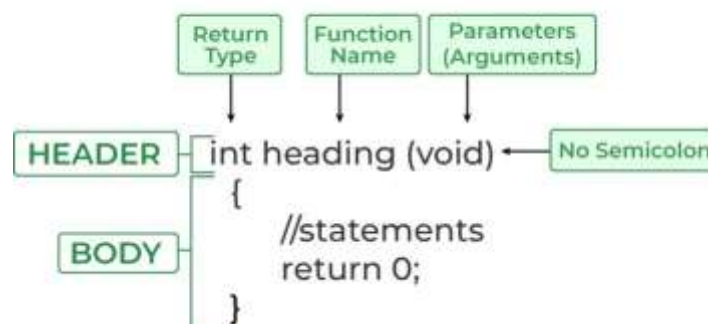


Function Declaration Note: A function in C must always be declared globally before calling it.

Function Definition

The function definition consists of actual statements which are executed when the function is called (i.e. when the program control comes to the function). A C function is generally defined and declared in a single step because the function definition always starts with the function declaration

Function Definition

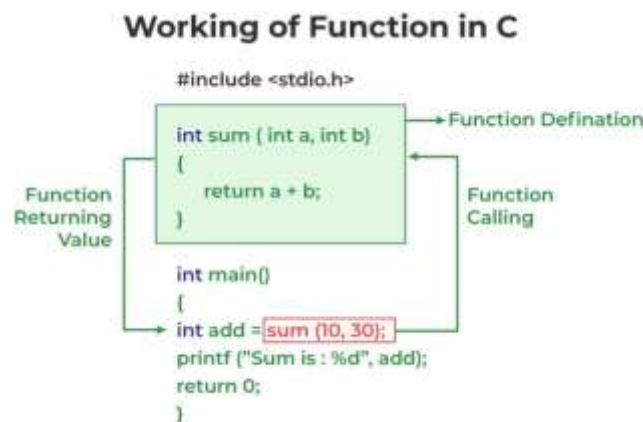


so we do not need to declare it explicitly. The below example serves as both a function definition and a declaration.

```
return_type function_name (para1_type para1_name, para2_type para2_name)  
{  
    // body of the function  
}
```

Function Call

A function call is a statement that instructs the compiler to execute the function. We use the function name and parameters in the function call. In the below example, the first sum function is called and 10,30 are passed to the sum function. After the function call sum of a and b is returned and control is also returned back to the main function of the program.



Working of function in C

Note: Function call is necessary to bring the program control to the function definition. If not called, the function statements will not be executed.

Problem Statement:

Implement a program to manage student records using structure and functions:

1. Static Memory Allocation:

- Use an array of structures to store n employee.
- Each record must include: emp_id, Name, salary.
- Use functions for:
 - Taking input
 - Displaying records

2. Dynamic Memory Allocation:

- Use pointers and `malloc()` to allocate memory for n employee records at runtime.
- Use similar functions as above for operations.

Conclusion:

This experiment provides practical exposure to:

- Structuring data efficiently with structures.
- Dynamic memory management.
- Writing modular and reusable code using functions.
- Real-world scenarios like inventory and student management.

Observations:

Feature	Remarks
Grouping Data	Structures allow logical grouping of related information
Static vs Dynamic	Static is fixed-size, dynamic supports flexible, runtime allocation
Modularity	Functions help in separating concerns and simplifying logic
Function Integration	Enables logical operations on structured data (e.g., display, calculate)
Reusability	Functions can be reused across different parts of the program

References –**a. Textbook –**

1. "The C Programming Language" by Brian W. Kernighan and Dennis M. Ritchie
2. "Data Structures and Algorithm Analysis in C" by Mark Allen Weiss
3. "Let Us C" by Yashavant Kanetkar

b. Online references –

1. GeeksforGeeks - Structures in C
<https://www.geeksforgeeks.org/structures-c/>
2. *TutorialsPoint - Structures in C*
https://www.tutorialspoint.com/cprogramming/c_structures.html
3. *Programiz - C Structures*
<https://www.programiz.com/c-programming/c-structures>
4. *W3Schools - C Structures*
https://www.w3schools.com/c/c_structs.php

Expected Oral Questions –

1. What is a structure in C, and why is it used?
2. How are members of a structure accessed in a C program?
3. What is the difference between an array and a structure?
4. Explain the memory allocation process for structures in C.
5. Can you nest one structure within another? If yes, how?
6. What are the limitations of using structures in C?
7. How do structures differ from classes in object-oriented programming languages?
8. What is Linear Regression algorithm?
9. What is a function, and why is it used in programming?
10. Explain the difference between a function definition and a function call.
11. What are the different types of functions (e.g., user-defined, built-in) in the language you are using?
12. How does parameter pass work in functions? What is the difference between passing by

value and passing by reference?

13. What is recursion, and can you give an example of a problem that can be solved using recursion?
14. How does the scope of variables work within functions?
15. What is the significance of the return type in a function?

FAQ's in Interview –

1. Q: What is a structure in C?

A: A structure is a user-defined data type in C that allows the grouping of variables of different data types under a single name.

2. Q: How do you initialize a structure in C?

A: A structure can be initialized by assigning values to its members individually or using an initializer list at the time of declaration.

3. Q: Can structures contain functions in C?

A: No, structures in C cannot contain functions, but they can contain function pointers.

4. Q: How is memory managed for a structure in C?

A: Memory for a structure is allocated based on the size of its members, and each member is stored in contiguous memory locations.

5. Q: Can you have an array of structures?

A: Yes, you can have an array of structures, where each element of the array is a structure.

6. Q: What is a function in programming?

A: A function is a block of code designed to perform a specific task, and it can be executed by calling its name. Functions help in organizing code, improving readability, and reusability.

7. Q: How do you pass arguments to a function?

A: Arguments are passed to a function through its parameters, which are defined in the function signature. They can be passed by value or by reference, depending on the programming language.

8. Q: What is recursion?

A: Recursion is a programming technique where a function calls itself to solve smaller instances of the same problem until a base condition is met.

9. Q: What is the difference between local and global variables?

A: Local variables are declared within a function and are only accessible within that function, whereas global variables are declared outside any function and are accessible throughout the program.

10. Q: Can you explain the concept of a pure function?

A: A pure function is a function that, given the same input, always produces the same output and has no side effects (i.e., it does not modify any external state).

u can have an array of structures, where each element of the array is a structure.